Métodos basados en Lanczos para hallar valores propios en arquitecturas paralelas

Facundo Gil

1 Introducción

El método de Lanczos se utiliza de forma habitual para el cálculo de algunos valores y vectores propios de matrices grandes y dispersas. Su formulación clásica presenta problemas de escalabilidad en arquitecturas paralelas modernas, principalmente debido a los costos asociados a la comunicación y a las sincronizaciones globales.

En este trabajo se revisan variantes del método de Lanczos desarrolladas con el objetivo de mejorar su desempeño en arquitecturas paralelas. En particular, investigamos métodos *bloqueados* que buscan reducir la cantidad de sincronizaciones y iteraciones globales.

2 Subespacios de Krylov

En esta sección se presenta una breve introducción a los subespacios de Krylov, ya que el método de Lanczos está basado en estos subespacios resulta esencial tener al menos, una definición corta sin entrar en mucho detalle.

Un subespacio de Krylov se define a partir de una matriz y un vector inicial, y se construye mediante aplicaciones sucesivas de la matriz sobre dicho vector. De forma intuitiva, este procedimiento genera una familia de vectores que contienen información cada vez más rica sobre la acción de la matriz, sin necesidad de acceder a todos sus elementos ni de operar con ella de manera global. A medida que se incrementa el número de iteraciones, el subespacio resultante crece y captura una porción cada vez mayor del comportamiento del operador original.

Formalmente, un subespacio de Krylov es:

$$\mathcal{K}_m(A, b) = \operatorname{span}\{b,\ Ab,\ A^2b,\ \ldots,\ A^{m-1}b\} \tag{1}$$

La utilización de subespacios de Krylov resulta natural porque la aplicación repetida de la matriz tiende a amplificar las componentes asociadas a ciertos valores propios, en particular aquellos dominantes. Esto implica que, incluso para subespacios de dimensión relativamente baja, es posible obtener aproximaciones útiles de algunos valores y vectores propios del problema original. Esta propiedad es especialmente relevante cuando se busca únicamente una parte del espectro y no su totalidad.

Otra característica importante de los subespacios de Krylov es que su construcción se basa exclusivamente en productos matriz–vector. En el caso de matrices grandes y dispersas, esta operación es computacionalmente eficiente y evita el almacenamiento explícito de la matriz completa.

3 Método de Lanczos

3.1 Definición y fundamentos

El método de Lanczos es un algoritmo iterativo propuesto por Cornelius Lanczos en 1950 para calcular valores y vectores propios de matrices simétricas (o hermitianas), en particular cuando se busca un número reducido de valores propios extremos de matrices grandes y dispersas. En lugar de abordar el problema espectral completo, el método se basa en proyectar la matriz original sobre un subespacio de dimensión reducida que se construye de manera progresiva.

Este subespacio es un subespacio de Krylov generado a partir de un vector inicial y de aplicaciones sucesivas de la matriz, lo que permite concentrar la información asociada a los componentes espectrales

más relevantes. A medida que el proceso avanza, se obtiene una secuencia de vectores ortonormales que forman una base de este subespacio, evitando la necesidad de almacenar o manipular la matriz completa.

Como resultado de esta proyección, la matriz original queda representada por una matriz tridiagonal simétrica de tamaño mucho menor, cuya estructura preserva las propiedades espectrales dominantes del problema original y permite calcular aproximaciones eficientes a los valores y vectores propios buscados. El carácter iterativo del método hace posible controlar el costo computacional y detener el proceso una vez alcanzada la precisión deseada, lo que resulta especialmente adecuado en el contexto de problemas de gran escala.

3.2 Proyección sobre un subespacio de Krylov

El problema de valores propios es esencialmente la resolución para un vector x del problema:

$$Ax = \lambda x$$

Es decir, encontrar un vector x tal que al aplicar la transformación A, el mismo se ve escalado por el factor λ. Siendo $A \in \mathbb{R}^{n\times n}$ a medida que crece n, crecen también los costos computacionales, al resolver el problema completo. La idea del método de Lanczos (y de otros métodos basados en subespacios de Krylov) es, en vez de buscar $x \in \mathbb{R}^n$, buscar una aproximación de x dentro de un subespacio $\mathcal{S}$ de dimensión menor m, con $m \ll n$. Esto último nos lleva a detallar que significa "buscar" dentro de un subespacio. Cuando tenemos un subespacio $\mathcal{S}$ cualquier vector perteneciente a ese subespacio se puede escribir como:

$$x = Q_m y$$

Donde $Q_m = [q_1, \ldots, q_m]$ es una base ortonormal del espacio $\mathcal{S}$ y $y \in \mathbb{R}^m$ son las coordenadas del vector en esa base. Volviendo al problema original y usando la ecuación anterior, vamos a querer que se cumpla:

$$A(Q_m y) = \lambda(Q_m y)$$

Como el subespacio $\mathcal{S}$ es de menor dimensión que el original, es esperable que esto no se cumpla, ya que el vector x original no tiene porque estar necesariamente dentro de $\mathcal{S}$, en general vamos a querer evaluar un residuo del tipo:

$$r = A(Q_m y) - \lambda(Q_m y)$$

Este residuo mide el error cometido al restringir el problema al subespacio $\mathcal{S}$. El objetivo es elegir y y λ de forma tal que dicho error sea lo más pequeño posible dentro del subespacio considerado. Desde un punto de vista geométrico, esto implica que no debe existir ninguna dirección dentro de $\mathcal{S}$ a lo largo de la cual el residuo pueda reducirse aún más. En consecuencia, el residuo debe ser ortogonal al subespacio $\mathcal{S}$. Si tenemos una base ortonormal Q_m asociada al subespacio $\mathcal{S}$ entonces esta condición se expresa como:

$$Q_m^T r = Q_m^T(A(Q_m y) - \lambda(Q_m y)) = 0.$$

Como Q_m es una base ortonormal ($Q_m^T Q_m = I_m$) tenemos que:

$$Q_m^T A Q_m y = \lambda y$$

Entonces en nuestra aproximación al problema, los valores propios de la transformación $T_m = Q_m^T A Q_m$ son la aproximación a los valores propios y $x = Q_m y$ la aproximación a los vectores propios. Cuando la matriz $A \in \mathbb{C}^{n\times n}$ el procedimiento es el mismo suplantando Q_m^T por Q_m^*.

Con el procedimiento anterior, logramos reducir la dimensión del problema efectivamente, siendo este el primer paso para poder resolver el problema de valores propios cuando la dimensión de la matriz es muy grande. Ahora, no especificamos como construir T_m ni como hallar la base ortonormal Q_m, ambos problemas no triviales cuando trabajamos con dimensiones altas, acá es donde se define el método de Lanczos (hasta ahora lo que hicimos se conoce como el método de Rayleigh-Ritz).

El método de Lanczos introduce una forma particular y altamente eficiente de construir la base Q_m del subespacio de Krylov, aprovechando la simetría de la matriz A. En lugar de recurrir a una ortogonalización completa en cada iteración, el método genera una secuencia de vectores ortonormales mediante una recurrencia de tres términos, lo que permite construir la base de manera incremental y con un costo computacional reducido. Como consecuencia de esta construcción, la matriz proyectada $T_m = Q_m^T A Q_m$ resulta tridiagonal y simétrica, una estructura que preserva información espectral relevante del problema original y simplifica notablemente el cálculo de sus valores propios.

Estas particularidades hacen del método de Lanczos una herramienta especialmente atractiva para el tratamiento de matrices grandes y dispersas, ya que combina eficiencia computacional y bajo requerimiento de memoria, como contrapartida es necesario que la matriz A sea simétrica.

3.3 Recurrencia de tres términos

Partiendo del subespacio definido en 1 lo que queremos es construir una base ortonormal sobre el, para esto usualmente se usa el método de Gram-Schmidt, el cual construye la base deseada de manera iterativa. En 1 se detalla el procedimiento a seguir.

Algorithm 1: Gram-Schmidt para espacios de Krylov

Input: Matriz $A \in \mathbb{R}^{n\times n}$, vector inicial $b \in \mathbb{R}^n$, dimensión m
Output: Base ortonormal $\{q_1, q_2, \ldots, q_m\}$ de $\mathcal{K}_m(A, b)$

```
q_1 ← b/||b||;
for k = 1 to m − 1 do
    v_{k+1} ← A q_k;
    for j = 1 to k do
        h_{j,k} ← ⟨v_{k+1}, q_j⟩;
        v_{k+1} ← v_{k+1} − h_{j,k} q_j;
    h_{k+1,k} ← ‖v_{k+1}‖;
    q_{k+1} ← v_{k+1}/h_{k+1,k};
```

Este enfoque es directo y funciona, pero tiene varias desventajas que a la hora de trabajar con matrices de gran tamaño se vuelven demasiado restrictivas. El costo computacional del método resulta demasiado prohibitivo cuando lo analizamos en detalle, para cada iteración necesitamos ortogonalizar contra todos los vectores ya hallados (para mantener la base ortonormal es necesario eliminar las componentes de un nuevo vector a agregar en las direcciones de los vectores anteriores). Para cada iteración k se hacen $k-1$ productos internos y actualizaciones, esto tiene un costo de total $\mathcal{O}(m^2 n)$ siendo m la dimensión del espacio y n el tamaño de la matriz. En cuanto al uso de memoria también estamos en problemas ya que se requiere almacenar m vectores de dimensión n, por lo tanto el costo en almacenamiento es $m \times n$.

Finalmente, este método es poco aplicable debido a que es muy sensible a los errores de precisión en aritmética de punto flotante, donde existen errores de redondeo. Esto último se debe a que los vectores hallados son propensos a la pérdida de ortogonalidad debido a estos errores numéricos.

Afortunadamente, cuando la matriz A es simétrica, se pueden explotar propiedades adicionales que permiten simplificar drásticamente el método presentado en el Algoritmo 1. En particular, la simetría de A implica que el proceso de ortogonalización puede reducirse significativamente, en lugar de ortogonalizar un nuevo vector candidato respecto de todos los vectores previamente construidos $\{q_1, \ldots, q_k\}$, resulta suficiente hacerlo únicamente respecto de los dos más recientes.

Más precisamente, al generar un nuevo vector candidato $v_{k+1} = Aq_k$, este resulta ortogonal a los vectores $\{q_1, \ldots, q_{k-2}\}$, por lo que solo es necesario eliminar sus componentes en las direcciones q_k y q_{k-1}. Esta propiedad, que se cumple en aritmética exacta para matrices simétricas, es la base teórica del método de Lanczos.

Entonces, el nuevo vector de la base se obtiene mediante la relación

$$\tilde{q}_{k+1} = v_{k+1} - \langle v_{k+1}, q_k\rangle q_k - \langle v_{k+1}, q_{k-1}\rangle q_{k-1},$$

La expresión anterior constituye la recurrencia de tres términos, donde llamamos $\alpha_k = \langle Aq_k, q_k\rangle$, y $\beta_{k-1} = \langle Aq_k, q_{k-1}\rangle$. El paso siguiente es normalizar el vector $\tilde{q}_{k+1}$ y obtener $q_{k+1} = \frac{\tilde{q}_{k+1}}{||\tilde{q}_{k+1}||} = \frac{\tilde{q}_{k+1}}{\beta_k}$. Ese desarrollo da lugar a la *recurrencia de tres términos*:

$$Aq_k = \beta_{k-1}q_{k-1} + \alpha_k q_k + \beta_k q_{k+1}$$

Observar que todos los vectores candidatos Aq_k pertenecen al subespacio de Krylov, mas formalmente:

$$Aq_k \in \mathcal{K}_{k+1}(A, b)$$

También hay que observar que las operaciones entre vectores de un subespacio dan resultados dentro del subespacio (la multiplicación por un escalar también), por lo tanto el método de Lanczos puede verse como tomar sucesivamente los vectores del subespacio de Krylov, forzar la ortogonalidad y normalizar.

3.4 Matriz tridiagonal

Volviendo al problema original, ya vimos que en el desarrollo de Rayleigh-Ritz, la aproximación a los valores propios esta en la transformación $T_m = Q_m^T A Q_m$, es decir la proyección de la matriz A sobre sobre el subespacio de Krylov (definido en 1). Mencionamos también que en el método de Lanczos esta matriz es *tridiagonal*, esto se puede ver facilmente usando la recurrencia que planteamos en la sección anterior.

Las entradas de la matriz T_m cumplen $(T_m)_{ij} = q_i^T A q_j$, entonces llegamos a una expresión en tres términos para Aq_k. Suplantando en la expresión para $(T_m)_{ij}$:

$$(T_m)_{ij} = q_i^T(\beta_{j-1}q_{j-1} + \alpha_j q_j + \beta_j q_{j+1})$$

Agrupando los productos internos tenemos que:

$$(T_m)_{ij} = \beta_{j-1}q_i^T q_{j-1} + \alpha_j q_i^T q_j + \beta_j q_i^T q_{j+1}$$

Luego tenemos que observar que los vectores q_k pertenecen a una base ortonormal, por lo tanto:

$$q_i^T q_\ell = \begin{cases} 1, & \text{si } i = \ell, \\ 0, & \text{si } i \neq \ell. \end{cases}$$

Entonces $(T_m)_{ij} \neq 0$ solo para $i = j-1$, $i = j$ o $i = j+1$, lo que implica que T_m es una matriz tridiagonal. También se puede notar que los valores de la matriz T_m son conocidos, ya que que el producto entre $q_i^T q_j$ solo puede valer uno o cero esto nos deja con la siguiente estructura para la matriz T_m:

$$T_m = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdots & 0 \\ \beta_1 & \alpha_2 & \beta_2 & \ddots & \vdots \\ 0 & \beta_2 & \alpha_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{m-1} \\ 0 & \cdots & 0 & \beta_{m-1} & \alpha_m \end{pmatrix}.$$

3.5 Esquema general del método de Lanczos clásico

El método de Lanczos clásico construye de manera iterativa una base ortonormal del subespacio de Krylov asociado a la matriz A y a un vector inicial b, utilizando la recurrencia de tres términos que explota la simetría de la matriz A. Esta construcción permite proyectar el problema espectral original sobre un subespacio de dimensión reducida, dando lugar a una matriz tridiagonal $T_m = Q_m^T A Q_m$, cuyo problema de valores propios es computacionalmente mucho más sencillo de resolver.

Los valores propios de T_m proporcionan aproximaciones a los valores propios de A, mientras que los vectores $Q_m y$ constituyen las aproximaciones a los vectores propios del problema original. Entonces ahora solo queda plantear un algoritmo para hallar T_m, y hallar sus valores propios.

Algorithm 2: Método de Lanczos clásico

Input: Matriz simétrica $A \in \mathbb{R}^{n\times n}$, vector inicial $b \neq 0$, número de iteraciones m
Output: Matriz tridiagonal T_m, base ortonormal $Q_m = [q_1, \ldots, q_m]$

```
q_1 ← b/‖b‖;
q_0 ← 0;
β_0 ← 0;
for k = 1 to m do
    v ← A q_k;
    α_k ← ⟨q_k, v⟩;
    r ← v − α_k q_k − β_{k−1} q_{k−1};
    β_k ← ‖r‖;
    if β_k = 0 then
        break;
    q_{k+1} ← r/β_k;
Construir T_m con α_k en la diagonal principal y β_k en las sub y superdiagonales;
```

El algoritmo presentado en 2 supone una formulación mas bien teórica ya que existen numerosos problemas que surgen a partir de este algoritmo, y es necesario entender que las bibliotecas que implementan el método de Lanczos no usan esta versión tan directa. Estos problemas surgen incluso en una implementación secuencial del algoritmo, es decir, no estan relacionados con las arquitecturas paralelas sino que tienen mas que ver con la suposición de precisión infinita (el algoritmo planteado en 2 viene a ser un modelo en aritmética exacta).

3.6 Limitaciones del algoritmo de Lanczos clásico

En esta sección vamos a ver algunos de los problemas que podrían surgir al implementar el algoritmo presentado en 2, en un sistema de precisión finita. En una implementación real, los errores de redondeo afectan en particular a las operaciones de producto interno y normalización, lo que nos lleva al principal problema Lanczos tal y como lo presentamos, la pérdida de ortogonalidad de los vectores hallados.

En el Algoritmo 2, la ortogonalización del nuevo vector candidato se realiza mediante la recurrencia

$$r \leftarrow v - \alpha_k q_k - \beta_{k-1} q_{k-1},$$

donde $v = Aq_k$. Esta operación es suficiente para garantizar ortogonalidad global únicamente en aritmética exacta, ya que bajo esa hipótesis el vector v pertenece exactamente al subespacio $\mathcal{K}_{k+1}(A, b)$ y es ortogonal a $\{q_1, \ldots, q_{k-2}\}$. En precisión finita, los errores de redondeo introducidos en el cálculo de v generan componentes no despreciables en direcciones anteriores que no son eliminadas por la recurrencia de tres términos.

Como consecuencia, los vectores $\{q_k\}$ dejan de formar una base ortonormal y comienzan a desarrollar componentes significativas en direcciones ya generadas. Este efecto se acumula iteración tras iteración y provoca que el proceso vuelva a amplificar direcciones asociadas a valores propios dominantes ya convergidos.

En aritmética finita ocurre que $Q_m^T Q_m \neq I$ y por lo tanto T_m deja de ser una proyección de A, lo que nos deja con una matriz que no representa el espectro de A. Este problema es estructural del algoritmo, hay que recordar que el mismo esta construido sobre la suposición de que $\langle q_i, q_j \rangle = 0 \quad \forall i \neq j$, sin esta suposición no existe la recurrencia de tres términos, la consecuencia de este problema es que aparecen valores propios espurios en el resultado final, o se repiten valores propios. Así mismo, la pérdida de ortogonalidad no tiene que ver solo con errores de redondeo, sino también con la convergencia de valores propios, que induce que el algoritmo vuelva a introducir componentes en direcciones ya convergidas (esto último se detalla en [Pai71] capitulo 8 "Error Behaviour & Convergene of the Symmetric Lanczos Process").

Todas las implementaciones prácticas del algoritmo tienen en cuenta la pérdida de ortogonalidad, ya que sin un control de este fenómeno el espectro resultante mediante el método se aleja del espectro deseado. Por este motivo, el algoritmo de Lanczos clásico rara vez se utiliza en su forma pura, y en su lugar se emplean variantes que incorporan mecanismos para mitigar este problema, como reortogonalización, técnicas de reinicio o modificaciones estructurales del método.

Otro efecto a notar es que si dejamos que el algoritmo 2 corra indefinidamente, entonces el tamaño del subespacio de Krylov m crece sin control. En cada iteración se agrega un nuevo vector $v_k = Aq_k$, lo que implica un crecimiento continuo del subespacio generado y, en consecuencia, del tamaño de la matriz tridiagonal T_m. Este crecimiento no solo incrementa los requerimientos de memoria, sino que también encarece el costo del post-procesamiento espectral asociado a T_m.

Permitir que m crezca indefinidamente resulta inviable en la práctica, tanto por razones numéricas como computacionales. Por un lado, la pérdida de ortogonalidad se agrava a medida que el subespacio crece, por otro, el almacenamiento de los vectores de la base y la manipulación de una matriz T_m cada vez más grande se vuelven prohibitivos. Esto hace necesario introducir mecanismos que limiten el tamaño efectivo del subespacio y concentren el esfuerzo computacional en la información espectral relevante.

3.6.1 Reinicios y re-ortogonalización

Los mecanismos de re-ortogonalización y reinicio atacan problemas distintos del algoritmo de Lanczos clásico, aunque en la práctica suelen emplearse de manera conjunta. La re-ortogonalización tiene como objetivo corregir la pérdida de ortogonalidad entre los vectores de la base, mientras que el reinicio introduce un control explícito sobre el crecimiento del subespacio de Krylov y del tamaño del problema reducido.

La re-ortogonalización consiste en imponer ortogonalidad adicional al nuevo vector generado, típicamente proyectándolo sobre un subconjunto o sobre la totalidad de los vectores previamente calculados. Este procedimiento permite mitigar la aparición de componentes espurias en direcciones ya generadas y reduce la repetición de valores propios convergidos. Sin embargo, su costo computacional crece rápidamente con el tamaño del subespacio, ya que requiere productos internos y comunicaciones adicionales, y además rompe la estructura simple de la recurrencia de tres términos que caracteriza al método de Lanczos.

Por otro lado, el reinicio actúa a un nivel más global del algoritmo. En lugar de permitir que el subespacio de Krylov crezca indefinidamente, el proceso se detiene luego de un número fijo de iteraciones y se reinicia utilizando la información espectral relevante obtenida hasta ese momento. De esta forma, se descartan direcciones asociadas a valores propios no deseados y se conserva únicamente la información necesaria para continuar refinando las aproximaciones buscadas.

Desde el punto de vista práctico, el reinicio no elimina la pérdida de ortogonalidad, pero limita sus efectos al evitar que estos se acumulen indefinidamente. En combinación con esquemas de re-ortogonalización parcial o selectiva, el reinicio permite construir algoritmos más robustos que mantienen controlados tanto el tamaño del subespacio como la degradación numérica de la base.

Estas ideas dan lugar a variantes modernas del método de Lanczos que difieren sustancialmente del esquema clásico presentado en el Algoritmo 2, y que constituyen la base de las implementaciones utilizadas en bibliotecas de software.

En su forma más simple, la re-ortogonalización implica reemplazar el cálculo de r por

$$r \leftarrow r - \sum_{j=1}^{k} \langle r, q_j \rangle q_j,$$

o bien por una versión selectiva donde la re-ortogonalización se aplica únicamente cuando se detecta una pérdida significativa de ortogonalidad. En este punto es importante notar que al hacer esto perdemos toda la ventaja de nuestro algoritmo, que aprovechaba la simetría de la matriz para justamente evitar hacer todos los productos internos durante la ortogonalización de la base. Si bien la re-ortogonalización no se hace en cada paso de la iteración, progresivamente tenemos que aplicar todos los productos internos lo que incrementa el costo computacional y vuelve el método mucho más pesado.

3.6.2 Reinicio implícito

En la práctica, el reinicio no consiste simplemente en detener el algoritmo y volver a comenzarlo con un nuevo vector inicial. La idea central es utilizar la información espectral contenida en la matriz tridiagonal T_m para modificar el subespacio de Krylov de forma tal que las componentes asociadas a valores propios no deseados se atenúen, mientras que las asociadas a los valores propios de interés se preserven.

Conceptualmente, esto equivale a aplicar un filtrado sobre el subespacio generado, el algoritmo se reorganiza para continuar el proceso en direcciones que contienen mayor información relevante del espectro, sin reconstruir explícitamente la base ni descartar toda la información acumulada. De esta manera, el método puede seguir refinando las aproximaciones buscadas sin permitir que el subespacio crezca indefinidamente.

Este tipo de reinicio permite controlar simultáneamente el tamaño del subespacio de Krylov y la acumulación de pérdida de ortogonalidad, manteniendo una estructura algorítmica cercana a la del método de Lanczos clásico. Este enfoque da lugar al denominado método de Lanczos con reinicio implícito, que es el utilizado en implementaciones, como la de la biblioteca *ARPACK*.

3.7 Limitaciones del método de Lanczos clásico en arquitecturas paralelas

Basándonos en el algoritmo presentado en 2, y teniendo en cuenta los problemas mencionados en la sección anterior podemos analizar que tan compatible es el algoritmo con las arquitecturas paralelas. Estas limitaciones no dependen de una implementación particular ni del tipo de arquitectura utilizada, sino que surgen directamente de la estructura del algoritmo y de las operaciones fundamentales que lo componen. Para evaluar la escalabilidad del método de Lanczos clásico en arquitecturas paralelas, resulta natural analizar las operaciones que se realizan en cada iteración del Algoritmo 2 y estudiar cómo podrían paralelizarse de manera directa.

En cada iteración, el costo computacional teóricamente está dominado por el producto matriz–vector $v = Aq_k$, seguido por el cálculo de productos internos para obtener los coeficientes α_k y β_k, la normalización del vector residual y, eventualmente, operaciones adicionales asociadas a mecanismos de re-ortogonalización o reinicio.

El producto matriz vector es altamente paralelizable, esto es debido a que la operación que se realiza es:

$$y_i = \sum_{j=1}^{n} A_{ij} x_j$$

Lo que nos deja con una suma independiente por filas, sin dependencia entre los diferentes cálculos. Además para cada fila se pueden paralelizar los cálculos para los productos $A_{ij}x_j$ haciendo sumas globales al final (reducciones de tipo suma). Esto se traduce a productos SpMV, donde la estructura es la misma solo que cambia la cantidad de operaciones que hacemos ya que solo se trabaja sobre las columnas no nulas (en SpMV tenemos problemas de acceso a memoria).

Recordemos que los coeficientes $\alpha_k = \langle q_k, Aq_k \rangle$ y $\beta_k = \|r\|$ requieren hacer productos internos, esta operación requiere hacer m multiplicaciones (siendo m el largo de los vectores) y luego una suma con los resultados de las mismas. Ambos coeficientes presentan la misma estructura, una etapa de multiplicaciones independientes (m productos independientes) las cuales tienen paralelización natural y luego los resultados deben ser acumulados en una suma. En este último paso se genera una dependencia global y una necesidad de sincronización de los eventuales hilos usados para paralelizar, limitando la escalabilidad del algoritmo.

Este problema se ve agravado cuando se consideran los mecanismos necesarios para controlar la pérdida de ortogonalidad y el crecimiento del subespacio de Krylov. La re-ortogonalización parcial o selectiva introduce productos internos adicionales, incrementando aún más la frecuencia de las sincronizaciones globales. Por su parte, los esquemas de reinicio, si bien son indispensables para la estabilidad numérica y el control del tamaño del subespacio, suelen requerir operaciones globales sobre la base y sobre la matriz tridiagonal asociada, lo que añade nuevos puntos de comunicación y coordinación entre procesos.

En resumen, para implementar el algoritmo de Lanczos con reinicios implicitos, deberíamos implementar las siguientes operaciones:

1. **Producto matriz–vector disperso (SpMV).** En cada iteración se calcula $v = Aq_k$. Esta operación es altamente paralelizable, por lo que su costo se ve limitado mas que nada por el acceso a memoria.

2. **Cálculo de coeficientes mediante productos internos.** Se computa

$$\alpha_k = \langle q_k, v \rangle,$$

lo que requiere un producto interno y, por lo tanto, una reducción global.

3. **Actualización del vector residual** Se construye el residual

$$r = v - \alpha_k q_k - \beta_k q_{k-1},$$

mediante operaciones vectoriales elemento a elemento, completamente paralelizables y sin sincronizaciones globales.

4. **Re-ortogonalización parcial o selectiva.** Para controlar la pérdida de ortogonalidad, se corrige el residual mediante

$$r \leftarrow r - \sum_{j \in \mathcal{I}} \langle q_j, r \rangle q_j,$$

lo que introduce productos internos adicionales y, por ende, nuevas reducciones globales.

5. **Cálculo de la norma del residual.** Se obtiene

$$\beta_{k+1} = \|r\|_2 = \sqrt{\langle r, r \rangle},$$

lo cual implica otra reducción global.

6. **Normalización y generación del siguiente vector de Lanczos.** Se define

$$q_{k+1} = \frac{r}{\beta_{k+1}},$$

mediante una operación vectorial paralelizable.

7. **Actualización de la matriz tridiagonal asociada.** Se almacenan los coeficientes α_k y β_k para construir la matriz tridiagonal T_k que representa la proyección del operador A sobre el subespacio de Krylov.

8. **Reinicio implícito del método.** Al alcanzar el tamaño máximo del subespacio, se aplican técnicas de reinicio implícito basadas en T_k para filtrar componentes espectrales no deseadas y definir el vector inicial del siguiente ciclo, lo que involucra operaciones globales y coordinación entre procesos.

9. **Verificación del criterio de convergencia.** Se evalúa la convergencia para decidir si el algoritmo finaliza o continúa con un nuevo ciclo.

Con este desgloce del algoritmo, vemos que las principales dificultades del método de Lanczos clásico en arquitecturas paralelas no están asociadas al producto matriz–vector ni a las actualizaciones vectoriales, que admiten paralelismo, sino a la frecuencia y naturaleza de las operaciones globales que el algoritmo requiere.

En particular, cada iteración del método involucra múltiples reducciones globales asociadas a productos internos y normas, necesarias para el cálculo de los coeficientes de recurrencia y para el control de la ortogonalidad. Estas reducciones introducen puntos de sincronización que limitan la escalabilidad del algoritmo a medida que aumenta el número de procesos.

Cuando se consideran los mecanismos necesarios para garantizar estabilidad numérica, como la re-ortogonalización parcial o selectiva, se incrementan aún mas la cantidad de productos internos por iteración, lo cual agrava el problema de sincronización entre hilos. Asimismo, los esquemas de reinicio implícito, si bien son indispensables para controlar el crecimiento del subespacio de Krylov y mejorar la robustez del método, requieren operaciones globales adicionales sobre la base de Lanczos y sobre la matriz tridiagonal asociada, introduciendo nuevos costos de comunicación y coordinación.

El rendimiento del método de Lanczos clásico en entornos paralelos queda dominado por la comunicación y la sincronización global, más que por el costo aritmético local. Esta observación motiva el desarrollo de variantes del método orientadas a reducir la frecuencia de las reducciones globales, y de comunicación, dando lugar a diferentes enfoques. A continuación vamos a revisar algunos métodos propuestos que atacan estos problemas.

4 Block Lanczos

4.1 Motivación y fundamentos

El método de Block Lanczos surge como una extensión natural del método de Lanczos clásico, diseñada específicamente para abordar las limitaciones de escalabilidad en arquitecturas paralelas que identificamos en la sección anterior. La idea fundamental consiste en reemplazar la construcción secuencial de vectores individuales por la generación simultánea de bloques de vectores, lo que permite reducir significativamente la frecuencia de las operaciones de comunicación global.

En lugar de trabajar con un único vector inicial b y construir iterativamente vectores individuales q_k, el método de Block Lanczos parte de una matriz inicial $B \in \mathbb{R}^{n\times p}$ cuyas columnas definen p direcciones iniciales linealmente independientes. A partir de esta matriz, el método genera bloques de vectores ortonormales que forman una base del subespacio de Krylov en bloques, definido como:

$$\mathcal{K}_m(A, B) = \text{span}\{B,\ AB,\ A^2B,\ \ldots,\ A^{m-1}B\} \tag{2}$$

Este subespacio generaliza la definición clásica y permite capturar información espectral de manera más rica desde las primeras iteraciones, ya que cada aplicación de A genera simultáneamente p nuevas direcciones que exploran el espacio de forma independiente.

Desde el punto de vista computacional, la ventaja central del enfoque en bloques radica en que las operaciones más costosas en términos de comunicación, como los productos internos y las normalizaciones, pueden realizarse de manera agrupada. En lugar de sincronizar los procesos para calcular un único producto interno por iteración, el método de Block Lanczos permite calcular múltiples productos internos en una sola fase de comunicación, reduciendo así el número total de sincronizaciones globales requeridas.

4.2 Recurrencia en bloques

De manera análoga al método de Lanczos clásico, donde la simetría de la matriz A permitía establecer una recurrencia de tres términos entre vectores individuales, en el método de Block Lanczos es posible derivar una recurrencia análoga que involucra bloques de vectores en lugar de vectores individuales.

Partiendo de una matriz inicial $B \in \mathbb{R}^{n\times p}$ con columnas linealmente independientes, el primer paso consiste en ortonormalizar este bloque para obtener $Q_1 \in \mathbb{R}^{n\times p}$ tal que $Q_1^T Q_1 = I_p$. Esta matriz Q_1 constituye el primer bloque de la base ortonormal del subespacio de Krylov en bloques.

A partir de este bloque inicial, el método genera iterativamente nuevos bloques mediante la aplicación de la matriz A. En cada iteración k, se calcula el bloque candidato:

$$V_{k+1} = AQ_k \tag{3}$$

Este bloque debe ser ortogonalizado respecto de los bloques previamente generados. En aritmética exacta y para matrices simétricas, la simetría del operador implica que V_{k+1} es ortogonal a todos los bloques $\{Q_1, \ldots, Q_{k-2}\}$, por lo que solo es necesario eliminar sus componentes en las direcciones de Q_k y Q_{k-1}.

Esta propiedad da lugar a la recurrencia en bloques:

$$AQ_k = Q_{k-1}B_{k-1}^T + Q_kA_k + Q_{k+1}B_k \tag{4}$$

donde:

- $A_k \in \mathbb{R}^{p\times p}$ es una matriz simétrica que contiene los coeficientes de proyección de AQ_k sobre Q_k,
- $B_k \in \mathbb{R}^{p\times p}$ es una matriz triangular superior que acopla el bloque actual con el siguiente,
- Q_{k+1} es el nuevo bloque ortonormal resultante.

Los coeficientes de estas matrices se obtienen mediante proyecciones:

$$A_k = Q_k^T AQ_k \tag{5}$$

$$\tilde{R} = V_{k+1} - Q_kA_k - Q_{k-1}B_{k-1}^T \tag{6}$$

donde $\tilde{R}$ es el bloque residual que contiene las componentes de V_{k+1} ortogonales a Q_k y Q_{k-1}. Finalmente, se realiza una factorización QR de este residual:

$$\tilde{R} = Q_{k+1} B_k \tag{7}$$

Esta factorización permite obtener simultáneamente el siguiente bloque ortonormal Q_{k+1} y los coeficientes B_k.

La recurrencia en bloques preserva la estructura algebraica del método de Lanczos clásico, pero opera sobre bloques de vectores en lugar de vectores individuales.

4.3 Estructura matricial resultante

Al igual que en el método de Lanczos clásico, donde la proyección del operador A sobre el subespacio de Krylov generaba una matriz tridiagonal, en el método de Block Lanczos la proyección resulta en una matriz con estructura tridiagonal en bloques.

Formalmente, si se generan m bloques de tamaño p, la matriz proyectada tiene la forma:

$$T_m = \begin{pmatrix} A_1 & B_1 & 0 & \cdots & 0 \\ B_1^T & A_2 & B_2 & \ddots & \vdots \\ 0 & B_2^T & A_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & B_{m-1} \\ 0 & \cdots & 0 & B_{m-1}^T & A_m \end{pmatrix} \in \mathbb{R}^{mp \times mp} \tag{8}$$

donde cada $A_k \in \mathbb{R}^{p \times p}$ es simétrica y cada $B_k \in \mathbb{R}^{p \times p}$ es triangular superior. Esta estructura es una generalización directa de la matriz tridiagonal escalar del caso clásico. Mientras que en el método de Lanczos con $p = 1$ los elementos de la diagonal y las subdiagonales son escalares (α_k y β_k), en el método de Block Lanczos con $p > 1$ estos elementos se convierten en matrices completas de tamaño $p \times p$. La matriz T_m preserva las propiedades espectrales fundamentales del operador original A dentro del subespacio considerado. Los valores propios de T_m proporcionan aproximaciones a los valores propios de A, y los vectores propios correspondientes pueden expandirse en la base de bloques $[Q_1, Q_2, \ldots, Q_m]$ para obtener aproximaciones a los vectores propios de A.

Una diferencia importante respecto al caso escalar es que la estructura de T_m es más densa, ya que cada bloque diagonal A_k contiene p^2 elementos en lugar de un único escalar. Sin embargo, el número de bloques necesarios para alcanzar una precisión dada suele ser significativamente menor que el número de iteraciones requeridas en el método clásico, especialmente cuando el espectro de A presenta agrupaciones de valores propios.

4.4 Esquema algorítmico

El algoritmo de Block Lanczos puede formularse de manera análoga al Algoritmo presentado para el método clásico, pero operando sobre bloques de vectores en lugar de vectores individuales. A continuación se presenta el esquema general:

Algorithm 3: Método de Block Lanczos

Input: Matriz simétrica $A \in \mathbb{R}^{n\times n}$, matriz inicial $B \in \mathbb{R}^{n\times p}$ con columnas linealmente independientes, número de iteraciones m

Output: Matriz tridiagonal en bloques $T_m \in \mathbb{R}^{mp\times mp}$, base ortonormal $\mathcal{Q}_m = [Q_1, Q_2, \ldots, Q_m]$

$Q_1 \leftarrow \mathrm{QR}(B)$ (tomar factor Q);
$Q_0 \leftarrow 0$;
$B_0 \leftarrow 0$;
for $k = 1$ **to** m **do**
  $V \leftarrow AQ_k$;
  $A_k \leftarrow Q_k^T V$;
  $R \leftarrow V - Q_k A_k - Q_{k-1} B_{k-1}^T$;
  $(Q_{k+1}, B_k) \leftarrow \mathrm{QR}(R)$;
  if $\|B_k\|$ *es suficientemente pequeño* **then**
    break;
Construir T_m con bloques A_k en la diagonal y B_k en las sub y superdiagonales;

Este esquema preserva la estructura fundamental del método de Lanczos clásico, pero todas las operaciones se realizan sobre bloques completos. Cada iteración genera p nuevas direcciones ortonormales simultáneamente, en contraste con el método clásico que genera una única dirección por iteración.

4.5 Ventajas en arquitecturas paralelas

El método de Block Lanczos promete mejoras sustanciales en términos de escalabilidad paralela respecto al método clásico, principalmente debido a la reducción en la frecuencia de las operaciones de comunicación global.

4.5.1 Reducción de sincronizaciones globales

La ventaja más significativa del método en bloques es que agrupa múltiples productos internos en una única operación de comunicación. En el método de Lanczos clásico, cada iteración requiere al menos dos reducciones globales: una para calcular $\alpha_k = \langle q_k, Aq_k\rangle$ y otra para calcular $\beta_k = \|r\|$.

En Block Lanczos, el cálculo de la matriz $A_k = Q_k^T A Q_k$ involucra p^2 productos internos entre columnas de Q_k y AQ_k, pero todos estos productos pueden calcularse localmente en cada proceso y comunicarse en una única fase de reducción global. De manera similar, la factorización QR del bloque residual puede implementarse de forma que minimice las sincronizaciones.

Si consideramos m iteraciones en bloques con tamaño de bloque p, el método clásico requeriría aproximadamente $2mp$ reducciones globales para generar el mismo subespacio, mientras que Block Lanczos requiere del orden de m reducciones. Esta reducción de un factor p en el número de sincronizaciones globales tiene un impacto directo en la escalabilidad del algoritmo, especialmente en sistemas con alta latencia de comunicación.

4.5.2 Uso eficiente de BLAS nivel 3

Las operaciones dominantes en Block Lanczos son productos matriz-matriz en lugar de productos matriz-vector. Específicamente:

- El cálculo de $V = AQ_k$ es un producto de una matriz dispersa por una matriz densa de tamaño $n \times p$
- La proyección $A_k = Q_k^T V$ es un producto matriz-matriz denso de tamaño $p \times n$ por $n \times p$
- La ortogonalización $R = V - Q_k A_k - Q_{k-1} B_{k-1}^T$ involucra productos matriz-matriz densos

Estas operaciones pueden implementarse mediante rutinas BLAS nivel 3 , que alcanzan una eficiencia computacional significativamente mayor que las rutinas BLAS nivel 2 utilizadas en el método clásico.

4.6 Limitaciones y consideraciones prácticas

Si bien el método de Lanczos por bloques esta pensado para atacar problemas de comunicación y sincronización en la práctica presenta problemas que limitan su aplicabilidad y robustez.

4.6.1 Elección del tamaño de bloque

El parámetro p (tamaño de bloque) tiene un impacto significativo en el rendimiento del algoritmo. Un valor muy pequeño de p no aprovecha suficientemente las ventajas del método en bloques, mientras que un valor muy grande incrementa el costo de las operaciones locales y puede generar subespacios que contienen información redundante.

En la práctica, el tamaño de bloque óptimo depende de múltiples factores:

- El número de valores propios buscados
- La estructura espectral de la matriz A
- Las características de la arquitectura paralela (número de procesos, latencia de comunicación)
- El costo relativo entre operaciones aritméticas y comunicación

4.6.2 Re-ortogonalización en bloques

Al igual que en el método clásico, la pérdida de ortogonalidad debido a errores de redondeo afecta al método de Block Lanczos. Sin embargo, en este caso la re-ortogonalización debe aplicarse no solo entre bloques completos, sino también dentro de cada bloque para garantizar que las columnas de Q_k permanezcan ortonormales entre sí.

Esto incrementa el costo computacional de las fases de re-ortogonalización y añade sincronizaciones adicionales cuando se implementa en paralelo. Las estrategias selectivas de re-ortogonalización resultan particularmente importantes en este contexto para mantener controlado el overhead asociado.

4.6.3 Pérdida de independencia lineal

Uno de los problemas específicos del método en bloques es la pérdida de independencia lineal dentro de un bloque durante las iteraciones. Cuando un valor propio de A converge, las columnas del bloque residual R pueden volverse linealmente dependientes, lo que resulta en una matriz B_k singular o numéricamente mal condicionada.

Este fenómeno no tiene un análogo directo en el método clásico, donde la convergencia de un valor propio simplemente lleva a $\beta_k \approx 0$. En el caso de bloques, la factorización QR del residual puede volverse inestable cuando las columnas de R son casi dependientes.

Esta pérdida de independencia lineal no es un efecto numérico aislado ni un problema menor de implementación, sino una consecuencia directa de la convergencia de valores propios dentro de un bloque. El método de Lanczos por bloques asume implícitamente que el bloque residual genera, en cada iteración, un conjunto de nuevas direcciones linealmente independientes. Cuando esta hipótesis deja de cumplirse, la estructura de la recurrencia en bloques se rompe y la factorización QR del residual se vuelve numéricamente inestable. Continuar el proceso requiere introducir mecanismos adicionales como deflación (sacar valores ya encontrados del problema original y continuar con el resto en un problema que solo contenga a los no hallados), reducción dinámica del tamaño del bloque o reconfiguración explícita del subespacio, lo que elimina la simplicidad del esquema original y limita seriamente su viabilidad como método general en implementaciones robustas y reutilizables.

En resumen si bien el método de Lanczos por bloques resulta conceptualmente atractivo para reducir la comunicación en arquitecturas paralelas, estas dificultades estructurales hacen que su uso directo sea poco frecuente en bibliotecas numéricas modernas, que optan por esquemas más flexibles de manejo del subespacio.

5 Block Krylov–Schur

El método de Krylov–Schur surge como una reformulación de los métodos de Arnoldi y Lanczos orientada a facilitar el reinicio, la deflación y el control estable del subespacio generado. A diferencia del

método de Lanczos por bloques, Krylov–Schur no impone una recurrencia fija entre bloques consecutivos, sino que trabaja con una base ortonormal general del subespacio de Krylov, sobre la cual se aplican proyecciones, reordenamientos espectrales y truncamiento controlado.

Esta flexibilidad es importante en el contexto de métodos en bloques. Mientras que en Lanczos por bloques la pérdida de independencia lineal dentro de un bloque rompe la estructura del algoritmo, en Krylov–Schur la convergencia parcial de valores propios se maneja de forma natural mediante la reordenación del subespacio y la eliminación explícita de direcciones ya convergidas. De este modo, el método evita la necesidad de introducir mecanismos como la reducción dinámica del tamaño del bloque o la deflación manual, preservando la estabilidad numérica y manteniendo un algoritmo mas robusto.

El método puede interpretarse como una reformulación del proceso de Lanczos en bloques que abandona la recurrencia rígida entre bloques consecutivos y trabaja directamente con un subespacio de Krylov general. En cada ciclo, el subespacio se expande mediante aplicaciones del operador y se proyecta el problema original sobre dicho subespacio para obtener aproximaciones espectrales. A partir de estas aproximaciones, el método reordena y trunca el subespacio, conservando únicamente las direcciones asociadas a los valores propios que aún Este mecanismo de reinicio evita los problemas de pérdida de independencia lineal característicos del método de Lanczos por bloques y permite manejar de forma estable la deflación y la convergencia parcial, manteniendo al mismo tiempo las ventajas del paralelismo de grano grueso propias de los métodos en bloques. En 4 se presenta el pseudocodigo a nivel conceptual.

Algorithm 4: Block Krylov–Schur

Input: A simétrica, bloque inicial $B \in \mathbb{R}^{n\times p}$, m (dimensión máx.), r (valores buscados)
Output: Aproximaciones a los valores propios buscados
$Q \leftarrow \mathrm{qr}(B)$;
while *no convergió* **do**

Expansión del subespacio (en bloques):
Construir una base ortonormal $Q = [Q_1, \ldots, Q_m]$ del subespacio generado por $B, AB, \ldots, A^{m-1}B$;

Proyección espectral:
$H \leftarrow Q^T AQ$;
Calcular aproximaciones espectrales resolviendo el problema reducido H;

Reinicio:
Seleccionar las direcciones asociadas a la información espectral de interés;
$Q \leftarrow QY$ y truncar el subespacio;
Usar el subespacio truncado como punto de partida del siguiente ciclo;

6 Experimentos

En este trabajo buscamos comparar el desempeño de los métodos que vimos anteriormente, con la idea de ver qué pasa cuando se los ejecuta en paralelo. El objetivo no es hacer un análisis teórico, sino mirar el comportamiento en la práctica: tiempos, cantidad de iteraciones y en qué partes del algoritmo se gasta más tiempo.

Los experimentos se realizan en mi computadora personal, utilizando únicamente 4 núcleos. En este informe nos referimos a paralelismo a pequeña escala (pocos procesos), suficiente para observar si el código aprovecha bien más de un núcleo y si aparecen costos asociados a la coordinación/ comunicación entre procesos.

6.1 Biblioteca usada

Para los experimentos buscamos mantener todos los algoritmos en el mismo ecosistema para poder hacer una comparación mas justa. La idea proviene de que al paralelizar cálculo hay muchas cosas que no dependen solo del algoritmo, entre otras decisiones prácticas que se toman al paralelizar cálculo:

1. cómo se guarda la matriz,
2. cómo se reparten los datos,

3. cuándo los procesos tienen que esperar a los demás,
4. cuántas veces todos tienen que comunicarse entre sí.

Por eso para enfocarnos en estudiar como se comportan los algoritmos fijamos el ecosistema y trabajamos con el mismo diseño para el cálculo paralelo.

6.1.1 Trilinos

Trilinos es una biblioteca escrita en `C++` pensada para trabajar con problemas numéricos grandes, en particular aquellos que involucran matrices dispersas. No es un programa único, sino un conjunto de librerías que se pueden usar de forma independiente según el tipo de problema que se quiera resolver.

La idea detrás de Trilinos es ofrecer herramientas ya implementadas para manejar matrices grandes, vectores y métodos iterativos, evitando tener que programar desde cero estructuras de datos y rutinas básicas que suelen ser complejas y propensas a errores.

En este trabajo se utiliza Trilinos como base para todas las pruebas, ya que permite resolver problemas de valores propios de tamaño grande y ejecutar el mismo código tanto en forma secuencial como en paralelo. De esta biblioteca usamos principalmente 2 paquetes: Anasazi y *Tpetra*.

6.1.2 Anasazi

Anasazi es uno de los paquetes de Trilinos dedicado específicamente a la resolución de problemas de valores propios, en particular para matrices grandes y dispersas. Su foco está en métodos iterativos de tipo Krylov, que permiten calcular solo una parte del espectro (por ejemplo, los valores propios de mayor módulo) sin necesidad de factorizar la matriz completa.

Entre los métodos implementados en Anasazi se encuentran variantes de Arnoldi, Lanczos y Krylov-Schur, tanto en versiones clásicas como en versiones por bloques. Estos métodos son especialmente adecuados para problemas de gran escala, donde los métodos directos resultan inviables por costos de memoria y tiempo de cómputo.

Una característica importante de Anasazi es que trabaja de forma abstracta con el operador del problema: no requiere acceder explícitamente a la matriz completa, sino únicamente a la operación producto matriz-vector. Esto permite desacoplar el método de la estructura concreta de almacenamiento de la matriz.

En este trabajo se utiliza el solver `BlockKrylovSchur`, que es una generalización del método Krylov-Schur al caso por bloques. Este enfoque permite mejorar la robustez numérica y, en ciertos casos, acelerar la convergencia, especialmente cuando se buscan varios valores propios cercanos entre sí.

Anasazi no maneja directamente la distribución de datos ni la paralelización de las estructuras algebraicas.

6.1.3 Tpetra

Tpetra es el paquete de Trilinos encargado de la representación y manipulación de vectores y matrices dispersas en memoria distribuida. Proporciona estructuras de datos eficientes y escalables, diseñadas para ejecutarse tanto en forma secuencial como paralela, utilizando MPI como modelo de paralelismo.

El concepto central en Tpetra es el de `Map`, que define cómo se distribuyen los índices globales de un vector o matriz entre los distintos procesos. A partir de este mapa, Tpetra construye objetos como `Vector`, `MultiVector` y `CrsMatrix`, garantizando que las operaciones algebraicas se realicen de forma consistente en un entorno paralelo.

En este trabajo, Tpetra se utiliza para construir el operador del problema de valores propios, almacenando la matriz dispersa y definiendo el producto matriz-vector requerido por Anasazi. De esta forma, el mismo código puede ejecutarse sin modificaciones tanto en un solo proceso como en múltiples procesos MPI.

6.1.4 Algoritmos comparados

A partir del solver `BlockKrylovSchur` vamos a comparar el algoritmo de Lanczos con reinicio implícito y una version blocked del mismo. Esto lo hacemos manejando la cantidad de bloques que usa el solver `BlockKrylovSchur` (manejado con el parametro `blockSize`):

- `blockSize = 1` : Algoritmo de Lanczos con reinicio Implicito (Krylov-Schur escalar)
- `blockSize > 1` : Algoritmo de Krylov-Schur *blocked*

Para cada version, ejecutamos cada algoritmo para diferente cantidad de núcleos y evaluamos como escala cada uno. En ambos tenemos cuidado de mantener el mismo tamaño del subespacio, de manera de obtener esencialmente el mismo algoritmo (recordar que la idea es evaluar el paralelismo).

6.2 Paralelización

La paralelización de los cálculos es manejada internamente por Trilinos. Utilizando `Tpetra::Map` se distribuyen la matriz y los vectores entre los distintos procesos, de forma que cada proceso almacena y opera únicamente sobre su parte local de los datos.

A partir de esta distribución, las operaciones se realizan en paralelo en cada proceso. La operación principal del algoritmo es el producto matriz–vector (SpMV). Aunque se utiliza un método bloque, este producto no cambia, en el caso bloque la matriz se aplica a varios vectores en forma simultánea, pero no se realiza un producto matriz–matriz general.

Por otro lado, las operaciones asociadas a la re-ortogonalización requieren información global, como los productos internos y las normalizaciones. Estas operaciones implican sincronización entre todos los procesos mediante comunicaciones colectivas, tal como se mencionó anteriormente.

6.2.1 Matriz utilizada

Para los experimentos se utiliza una matriz Laplaciana unidimensional discreta. Esta matriz tiene estructura tridiagonal, con valores constantes en la diagonal principal y en las diagonales adyacentes. En forma esquemática, la matriz es de la forma:

$$A = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix}.$$

Esta matriz se construye de manera directa en el código y es dispersa, ya que cada fila contiene solo unos pocos elementos distintos de cero. Además, es simétrica, lo que permite utilizar métodos iterativos adecuados para este tipo de problemas.

La elección de esta matriz se debe principalmente a que es un ejemplo simple, fácil de generar y verificar, pero que al mismo tiempo presenta características que resultan interesantes para el análisis del paralelismo. En particular, cuando el tamaño de la matriz es grande, los valores propios se encuentran muy cercanos entre sí en los extremos del espectro, lo que dificulta la convergencia de los métodos iterativos.

Por otro lado, la estructura dispersa de la Laplaciana hace que el producto matriz–vector tenga un patrón de comunicación sencillo en paralelo, lo que permite distinguir claramente entre el costo del cálculo local y el costo asociado a las operaciones que requieren sincronización global, como la re-ortogonalización.

6.2.2 Configuración usada para los experimentos

Los principales parametros modificados durante los experimentos fueron:

- `nGlobal`: tamaño de la matriz cuadrada del problema.
- `blockSize`: número de vectores que se avanzan simultáneamente en cada iteración del método.
- `numBlocks`: cantidad de bloques almacenados en la base de Krylov.

Los parametros `numBlocks` junto con `blockSize` determinan la dimensión máxima del subespacio usado para la búsqueda. Para todos los experimentos dejamos fijo `nGlobal = 200000`, y variamos las combinaciones de `numBlocks` y `blockSize` para obtener siempre la misma dimensión máxima del subespacio. En todos los casos se ejecutó el algoritmo para 1,2,3 y 4 núcleos, reportando los tiempos de ejecución con su desglose por operación, usando el reporte brindado por `Trilinos` (`TimeMonitor`).

7 Resultados

7.1 Lanczos clásico (`blockSize = 1`)

En este experimento se utilizó el método Lanczos clásico, equivalente a `blockSize = 1`. En la Tabla 1 se muestran los tiempos totales de ejecución para distintos números de núcleos.

Núcleos (p)	Tiempo solve [s]
1	363.7
2	161.7
3	138.7
4	111.6

Table 1: Tiempo de ejecución para Lanczos clásico.

A partir de estos tiempos se calculó el *speedup* $S_p = T_1/T_p$ y la eficiencia $E_p = S_p/p$, cuyos valores se presentan en la Tabla 2.

Núcleos (p)	Speedup	Eficiencia
1	1.00	1.00
2	2.25	1.12
3	2.62	0.87
4	3.26	0.82

Table 2: Speedup y eficiencia para Lanczos clásico.

7.2 Block Krylov–Schur (`blockSize = 4`)

En este experimento se utilizó el método Block Krylov–Schur con `blockSize = 4`, manteniendo constante la dimensión máxima del subespacio de búsqueda. En la Tabla 3 se muestran los tiempos de ejecución obtenidos.

Núcleos (p)	Tiempo solve [s]
1	830.5
2	456.0
4	296.9

Table 3: Tiempo de ejecución para Block Krylov–Schur.

En la Tabla 4 se muestran el speedup y la eficiencia correspondientes.

Núcleos (p)	Speedup	Eficiencia
1	1.00	1.00
2	1.82	0.91
4	2.80	0.70

Table 4: Speedup y eficiencia para Block Krylov–Schur.

8 Conclusiones

En este trabajo se revisaron variantes del método de Lanczos orientadas a mejorar su desempeño en arquitecturas paralelas, con particular énfasis en los métodos bloqueados. A partir del análisis teórico y de los experimentos realizados, se obtuvieron resultados que permiten comprender mejor las limitaciones prácticas de estos enfoques en escenarios de paralelismo a pequeña escala.

8.1 Principales hallazgos experimentales

Los experimentos realizados sobre una matriz Laplaciana de tamaño $n = 200000$ muestran que, en el contexto de ejecución considerado (4 núcleos), el método de Lanczos clásico con reinicio implícito presenta un mejor escalado paralelo que el método Block Krylov–Schur.

Para el caso de Lanczos clásico se obtuvo un speedup de 3.26 con 4 núcleos y una eficiencia de 0.82, mientras que Block Krylov–Schur alcanzó un speedup de 2.80 con una eficiencia de 0.70 para la misma cantidad de núcleos. Este resultado es contraintuitivo si consideramos únicamente el análisis teórico, ya que el método bloqueado fue diseñado específicamente para reducir las sincronizaciones globales y mejorar el paralelismo.

8.2 Interpretación de los resultados

El análisis del desglose de tiempos reportado por Trilinos revela que la etapa de ortogonalización domina el costo total de ejecución en ambos métodos. En el caso del método bloqueado, esta etapa resulta particularmente costosa debido a:

1. La necesidad de ortogonalizar bloques completos de vectores, lo que incrementa el número de operaciones locales.
2. El mayor volumen de comunicación entre procesos al realizar productos internos sobre bloques en lugar de vectores individuales.
3. La estructura más densa de la matriz tridiagonal por bloques resultante, que complica las operaciones de proyección y reinicio.

Estos factores introducen un overhead que, en el contexto experimental considerado, supera las ventajas teóricas asociadas a la reducción de sincronizaciones globales. Es importante destacar que estos resultados corresponden a un escenario específico: paralelismo con pocos núcleos (hasta 4) y un problema de tamaño intermedio. En arquitecturas con mayor número de procesos y problemas de mayor escala, donde el costo de comunicación tiene un impacto más pronunciado, es posible que las conclusiones sean diferentes.

8.3 Limitaciones del estudio

Los experimentos realizados presentan varias limitaciones que deben considerarse al interpretar los resultados:

- **Escala reducida**: Se trabajó únicamente con 4 núcleos, lo que representa un escenario de paralelismo muy limitado. En sistemas con decenas o cientos de procesos, el comportamiento relativo de los métodos podría cambiar significativamente.
- **Tipo de matriz**: Se utilizó únicamente una matriz Laplaciana tridiagonal. Otros tipos de matrices dispersas con patrones de comunicación más complejos podrían mostrar comportamientos diferentes.

8.4 Comentarios finales

Los métodos para resolver el problema de valores propios se enfrentan de una u otra forma al problema de la pérdida de la ortogonalidad cuando se implementan en aritmética de precisión finita. Esto lleva a buscar soluciones que preserven la ortogonalidad, lo que inevitablemente genera puntos de sincronización durante la ejecución del algoritmo, por lo tanto el principal desafío para la implementación en arquitecturas paralelas radica en la comunicación entre nodos y el uso eficiente de las instancias de sincronización.

References

[Pai71] Christopher C. Paige. *The Computation of Eigenvalues and Eigenvectors of Very Large Sparse Matrices.* PhD thesis, University of London, 1971. Available at https://www.cs.mcgill.ca/ chris/pubClassic/PaigeThesis.pdf.